

Implementing such changes would risk affecting the current behavior of the class `AnonymousSurvey`. For example, it's possible that while trying to allow each user to enter multiple responses, we could accidentally change how single responses are handled. To ensure we don't break existing behavior as we develop this module, we can write tests for the class.

Testing the `AnonymousSurvey` Class

Let's write a test that verifies one aspect of the way `AnonymousSurvey` behaves. We'll write a test to verify that a single response to the survey question is stored properly:

```
test_survey.py  from survey import AnonymousSurvey

❶ def test_store_single_response():
    """Test that a single response is stored properly."""
    question = "What language did you first learn to speak?"
❷    language_survey = AnonymousSurvey(question)
    language_survey.store_response('English')
❸    assert 'English' in language_survey.responses
```

We start by importing the class we want to test, `AnonymousSurvey`. The first test function will verify that when we store a response to the survey question, the response will end up in the survey's list of responses. A good descriptive name for this function is `test_store_single_response()` ❶. If this test fails, we'll know from the function name in the test summary that there was a problem storing a single response to the survey.

To test the behavior of a class, we need to make an instance of the class. We create an instance called `language_survey` ❷ with the question "What language did you first learn to speak?" We store a single response, English, using the `store_response()` method. Then we verify that the response was stored correctly by asserting that English is in the list `language_survey.responses` ❸.

By default, running the command **pytest** with no arguments will run all the tests that **pytest** discovers in the current directory. To focus on the tests in one file, pass the name of the test file you want to run. Here we'll run just the one test we wrote for `AnonymousSurvey`:

```
$ pytest test_survey.py
===== test session starts =====
--snip--
test_survey.py . [100%]
===== 1 passed in 0.01s =====
```

This is a good start, but a survey is useful only if it generates more than one response. Let's verify that three responses can be stored correctly. To do this, we add another method to `TestAnonymousSurvey`:

```
from survey import AnonymousSurvey

def test_store_single_response():
```